

プログラミング言語論

第3回

文の翻訳

C言語の文

表明

Hoare論理

篠埜 功

文の翻訳について

while文などは効率のよい機械語にほとんどそのままコンパイルされる。

while文
while E do S
の翻訳



選択文の翻訳

選択文(Cにおけるswitch文など)は、さまざまな実装方法があり得るので、実装によって使い方に影響がある。

(実装例1) ジャンプテーブル(配列)を用いたコードに変換し、連続した定数のみを分岐先指定に用いるように推奨する。

(実装例2) 分岐数が少ない場合(例えば7以下の場合)、条件分岐のネストにする。多い場合は、分岐先指定の定数がある程度密ならジャンプテーブル(配列)を用いる(例えば配列の半分以上が埋まる場合)。それ以外の場合ハッシュテーブルを用いる。

C言語の文の構文

C言語の文の構文は拡張BNF記法で以下のように定義される。

```
<stmt> ::= ;  
          | <exp>;  
          | { <stmt-list> }  
          | if (<exp>) <stmt> | if (<exp>) <stmt> else <stmt>  
          | while ( <exp> ) <stmt>  
          | do <stmt> while (<exp>) ;  
          | for (<opt-exp>; <opt-exp>; <opt-exp>) <stmt>  
          | switch ( <exp> ) <stmt>  
          | case <const-exp> : <stmt>  
          | default : <stmt>  
          | break; | continue; | return; | return <exp>;  
          | goto <label>; | <label> : <stmt>  
  
<stmt-list> ::= <stmt> *  
<opt-exp> ::= ε | <exp>
```

C言語について

C言語は、講義で使っているPascal言語等と以下のような違いがある。

- 基本型にブール型はない。0以外の値をtrue, 0をfalseとして扱う。
- 代入演算子は=, 比較演算子は==, !=である。
- while文の条件部分は括弧で囲まなければならない。
- begin, endの代わりに中括弧{, }を用いる。
- 論理andと論理or演算子は&&と||である。

短絡評価(short circuit evaluation)

C言語(やModula2, Pascal等)ではand, or式ついて短絡評価を行う。

短絡評価 --- 第2引数は必要な場合のみ評価。

(例)

```
x = 1;
```

```
if (x == 1 || y == 2)
```

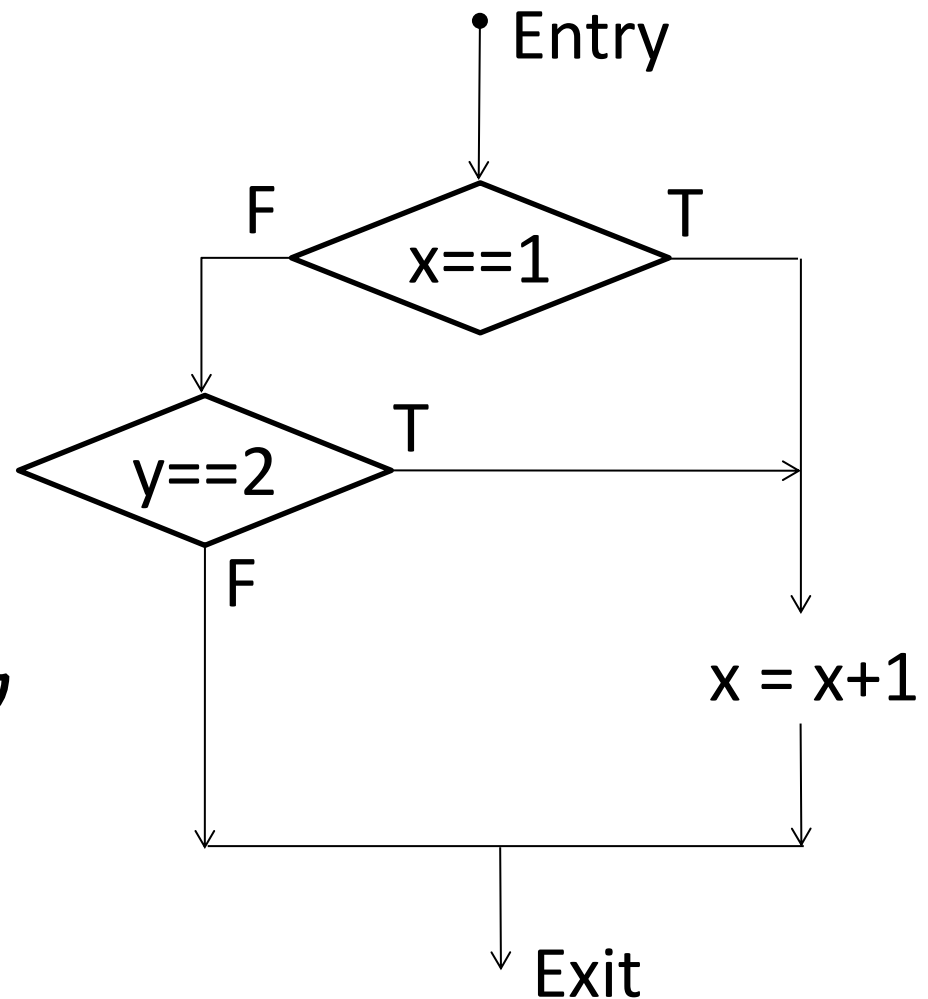
```
...
```

のような例の場合、 $x==1$ が真であり、if文の条件式は $y==2$ の値に係わらず真であるので、 $y==2$ を評価せずif文の本体を実行する。

短絡評価の場合の制御フロー

if (x==1 || y==2)
 x = x + 1;
の制御フロー

短絡評価により制御フローは複雑になるが、実行効率が良くなる。



参考

制御フローを図示したものはグラフ構造を成す。制御フローグラフは、goto文がない場合、複雑さが、ある尺度(木幅 tree-width)に関して一定値以下になることが知られている。

- goto文無しのAlgol, goto文無しのPascal --- 3以下
- Modula-2 --- 5以下 (Modula-2にはgoto文は無い)
- goto文無しのC --- 6以下

短絡評価がなければ上記の木幅は1ずつ減少。

Algol, Pascalに比べ、Modula-2はループについて複数個所での終了(break)を許し、関数も複数個所での終了(return)を許す。Cではさらにcontinue文がある。

(参考文献) Mikkel Thorup, “All structured programs have small tree-width and good register allocation”, *Information and Computation*, Vol. 142, pp. 159–181, 1998.

練習問題

(1) 以下のC言語のプログラム断片の制御フローを図示せよ。

```
if (y==1 || y==2)
    if (x > 0)
        x = x - 1;
```

(2) 以下のC言語のプログラム断片の制御フローを図示せよ。

```
while (x > 0) {
    if (x%2==0 || x%3==0)
        s = s + x;
    x = x - 1;
}
```

表明 (assertion)

表明(assertion) --- 条件式。

Javaでは表明を記述するための構文がある。

C++では、assert.hをインクルードすることにより、assertというマクロを表明として使うことができる。

(Javaの例)

```
int sum (int n) {  
    assert (n > 0);  
    ...  
}
```

sumというメソッドの呼び出しでは引数には正のint型の値しか渡されないという場合にこのようなassertionを記述しておけば、バグの発見に役立つ。

不変表明 (invariant)

プログラム中のある地点において、そこに制御が到達したときにはある条件式が必ず成り立つ場合、その条件式を不変表明 (invariant) という。

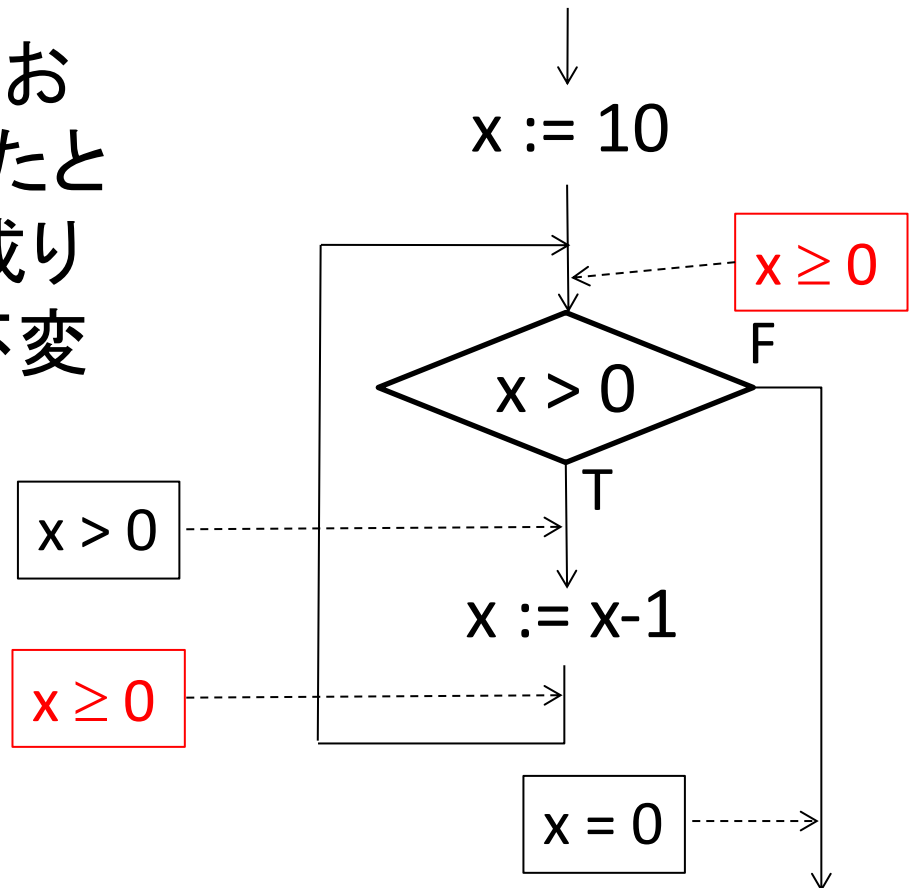
(例)

$x := 10;$

while $x > 0$ **do**

$x := x - 1$

特に、 $x \geq 0$ をループ不変表明 (loop invariant) という。



変数の値は整数
と仮定する

表明の例

この授業では、プログラム中で表明を{ }で囲んで記述する。

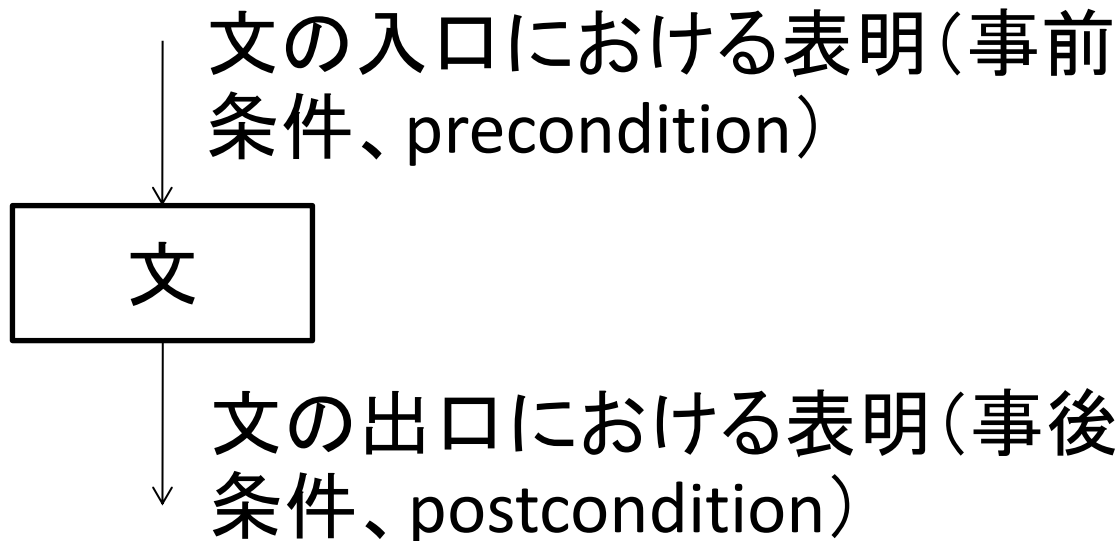
```
x := 10
{ x ≥ 0 }
while x > 0 do
  { x > 0 }
  x := x - 1
  { x ≥ 0 }
{ x = 0 }
```

左の4箇所の表明は不変
表明(invariant)

このようなことを証明する
ための論理体系がHoare
論理

事前条件、事後条件

Single entry/single exitの構成要素からなるプログラムの場合、プログラム中の文の振る舞いはその文の入口と出口における表明によって特徴付けることができる。(命令型言語のプログラム(文)の意味は、文の実行による状態の変化なので。)



Hoare triple

文の前後に表明を記述することにより命令型言語のプログラムの意味を記述できる。この考えを提唱したのがCharles Antony Richard Hoare(略してTony Hoare)である。表明を $\{ \}$ で囲んで表し、文の前後に書いたものをHoare tripleという。

Hoare triple
 $\{P\} S \{Q\}$
の意味

Pが成り立つ任意の状態 σ において文Sを実行し、状態 σ' で終了したら、 σ' においてQが成り立つ。

Hoare tripleの例

$\{ a = 0 \} \ a := a + 1 \ \{ a = 1 \}$

$a = 0$ が成り立っている状態で $a := a + 1$ を実行すると、実行後の状態では $a = 1$ が成り立つ。

$\{ a = 1 \} \ a := a - 1; a := a + 1 \ \{ a = 1 \}$

$a = 1$ が成り立っている状態で $a := a - 1; a := a + 1$ を実行すると、実行後の状態では $a = 1$ が成り立つ。

$\{ a = 5 \} \ \mathbf{while} \ a > 0 \ \mathbf{do} \ a := a - 1 \ \{ a = 0 \}$

$a = 5$ が成り立っている状態でこのwhile文を実行すると、実行後の状態では $a = 0$ が成り立つ。

部分正当性

Hoare tripleでは、文Sが停止することは言っていない（while文は停止しない場合もあるので）。停止性は別途示す必要がある。Hoare tripleは部分正当性を表している(partial correctness assertion)。

停止しない例: **while** true **do** x := 1

{true} **while** true **do** x := 1 {false} --- このHoare tripleは成立する。

(参考) 完全正当性 (total correctness)

--- 部分正当性 + 停止性

完全正当性の表明は、 $[P] S [Q]$ のように $[\]$ で表すこともある。

(参考) while ruleを拡張することにより、Hoare tripleがtotal correctnessを示すようにすることができる。

Hoare論理

文に対するHoare Tripleは、文の構造に従って規則的に構築することができる。各構文要素(construct)に対応する規則があり、それを集めたものをHoare論理(Hoare logic)という。Hoare論理は公理(axiom)から推論規則(inference rule)に従って導く形になっており、公理的意味論(axiomatic semantics)である。(公理的意味論には様々な形がある。)

(参考) flowchart上でFloydが同様のことを考えている。

(参考文献1) C. A. R. Hoare, "An axiomatic basis for computer programming", *Communications of the ACM*, 12(10):576–580, 583, 1969.

(参考文献2) R. W. Floyd, "Assigning meanings to programs", *Proceedings of the American Mathematical Society Symposium on Applied Mathematics*, Vol. 19, pp. 19–32. 1967.

Hoare論理の規則

$$\frac{\{P\} S_1 \{Q\} \quad \{Q\} S_2 \{R\}}{\{P\} S_1; S_2 \{R\}} \quad (\text{composition rule})$$

$$\frac{\{P \wedge E\} S_1 \{Q\} \quad \{P \wedge \neg E\} S_2 \{Q\}}{\{P\} \text{ if } E \text{ then } S_1 \text{ else } S_2 \{Q\}} \quad (\text{conditional rule})$$

$$\frac{\{P \wedge E\} S \{P\}}{\{P\} \text{ while } E \text{ do } S \{P \wedge \neg E\}} \quad (\text{while rule})$$

$$\frac{}{\{Q[E/x]\} x := E \{Q\}} \quad (\text{assignment axiom})$$

$$\frac{P \Rightarrow P' \quad \{P'\} S \{Q'\} \quad Q' \Rightarrow Q}{\{P\} S \{Q\}} \quad (\text{consequence rule})$$

例

Hoare triple $\{x \geq 0\} \text{ while } x > 0 \text{ do } x := x - 1 \{x = 0\}$
は成り立つが、これを規則を使って導くことができる。

$$\begin{array}{c} \frac{}{\quad} \text{(assignment)} \\ x \geq 0 \wedge x > 0 \Rightarrow x-1 \geq 0, \quad \{x-1 \geq 0\} x := x - 1 \{x \geq 0\} \\ \hline \quad \text{(consequence)} \\ \{x \geq 0 \wedge x > 0\} x := x - 1 \{x \geq 0\} \\ \hline \quad \text{(while)} \\ \{x \geq 0\} \text{ while } x > 0 \text{ do } x := x - 1 \{x \geq 0 \wedge \neg x > 0\}, \quad x \geq 0 \wedge \neg x > 0 \Rightarrow x = 0 \\ \hline \{x \geq 0\} \text{ while } x > 0 \text{ do } x := x - 1 \{x = 0\} \end{array}$$


(consequence)

(補足1) 1つの文に対して成り立つHoare tripleは複数存在する。

(補足2) consequence規則は一方だけの適用も可とする。

複雑な例

	$r := x;$	
	$q := 0;$	$\{ \neg y \leq r$
$\{\text{true}\}$	$\text{while } y \leq r \text{ do}$	$\wedge$
	begin	$x = r + y * q \}$
	$r := r - y; q := 1 + q$	
	end	

「このプログラムを実行すると、 x を y で割ったときの商と余りが q と r に格納された状態になる」ということをこのHoare tripleは表している。

このHoare tripleの導出は省略する。

Assertionについて

Assertionに何を書いてよいかを定める必要があるが、この講義では、true, false, 変数、整数、和(+), 差(-), 積(*), 大小比較(<, >, ≤, ≥), 等しさの判定(=), 論理演算(∧, ∨, ¬, ⇒)を使ってよいものとする。さらに∀, ∃を加えてもよいが、この講義(や試験)では上記範囲とする。算術演算、大小比較などに関する推論規則も定める必要があるが、本講義では通常の常識で推論を行ってよいものとする。

(参考文献1) Glynn Winskel, *"The formal semantics of programming languages"*, 1993, MIT Press. の第6章

(参考文献2) C. A. R. Hoare, "An axiomatic basis for computer programming", *Communications of the ACM*, 12(10):576–580, 583, 1969.

Hoare論理の練習問題

以下のHoare Tripleを推論規則を使って導け。

(1) $\{ a = 0 \} \ a := a + 2 \ \{ a = 2 \}$

(2) $\{ a = 3 \} \text{ if } a = 3 \text{ then } a := a + 1 \text{ else } a := a - 1 \ \{ a = 4 \}$

(3) $\{ a = 1 \} \text{ while } a < 5 \text{ do } a := a + 1 \ \{ a = 5 \}$

参考

基本データ型は範囲は有限であり、それに対応した規則が実際には必要である。

(例) 0以上の整数の型の場合のオーバーフローの扱い

(1) オーバーフローが起こるとプログラムがエラーで終了する。

$$\neg \exists x (x = \max + 1)$$

(2) オーバーフローの場合は最大値を値とする。

$$\max + 1 = \max$$

(3) 最大値での割り算の余りとする。

$$\max + 1 = 0$$